

PassiveSentinel: A Read-Only, Streaming, Multi-Head Threat-Intelligence Pipeline for One-Way Monitored Critical-Infrastructure Links

Architecture, layer-by-layer design, preprocessing and normalization, and benchmark protocol (SIH 2026 PS-145)

Authors: [add names, affiliations] | Draft manuscript v1.0 | 1 October 2026

Abstract. Operators of critical infrastructure often observe gateway and peering links through passive mirrors or hardware data diodes that copy traffic into a monitoring enclave in one direction only. Any analytics in that enclave must work from what it passively sees, and can never probe, complete a handshake with, or push a mitigation back to the traffic source. We present **PassiveSentinel**, a streaming pipeline that detects, classifies and scores six threat families (volumetric and protocol DDoS, botnet C2 beaconing, DGA and DNS tunnelling, malware in encrypted sessions, reconnaissance and port scanning, and data exfiltration) under three hard constraints: read-only ingest, no payload decryption, and incremental processing with bounded alert latency. The pipeline combines a deterministic preprocessing and normalization layer, specialist detectors, a benign-only autoencoder gate for unknown anomalies, and a compact unified encoder with typed Boolean, Choice and Score heads, inspired by the publicly described System One decision architecture, that fuses all evidence into calibrated alerts in a standardized schema. We specify a leakage-safe training and validation approach and a complete benchmark protocol (datasets, splits, metrics including per-class F1, macro-F1, PR-AUC, ECE and time-to-detect, throughput and latency methodology, and ablations). Numerical targets in this document are design goals; measured results are produced by the benchmark scripts of the accompanying repository and recorded in the result templates of Section 6.

Keywords: passive network monitoring, data diode, intrusion detection, NetFlow/IPFIX, encrypted traffic analysis, DGA, beaconing, calibrated multi-task learning, streaming ML.

Contents

1. Introduction	2
2. Threat Model and What Can Be Observed	2
3. System Overview	4
4. Layer-by-Layer Design	5
5. Training and Validation Approach	11
6. Benchmark Protocol	12
7. Ablation Study Design	13
8. Additional Treatments Considered	13
9. Limitations and Threats to Validity	14
10. Repository Layout and Reproducibility	14
11. Conclusion	15
References	15
Appendix A. Example alert record	16
Appendix B. Initial hyper-parameter defaults (tunable)	16

1. Introduction

Mirroring (SPAN/TAP) and hardware data diodes are widely used to observe critical links because they remove a whole class of attack: a compromised monitoring system cannot become a pivot into the production network, and the captured stream keeps a clean chain of custody for forensics. The price is that the intelligence layer is *blind to reply*. It cannot send probes, cannot complete a TCP or TLS handshake to test a suspicious host, and cannot issue a block. Everything must be inferred from packet captures, exported flow records (NetFlow, IPFIX, sFlow) and derived metadata.

This paper describes a complete design that respects those limits and still reaches strong, explainable detection across six threat families. We make four contributions:

- **A constraint-first architecture** in which read-only ingest, no decryption and streaming are structural properties (Section 3), not options.
- **A layer-by-layer specification** (L0 to L9) giving, for each layer, its purpose, operation, design rationale and output, including full preprocessing and normalization (Sections 4.2 to 4.5).
- **A unified typed-head encoder** (L6) that fuses specialist evidence into one calibrated decision with Boolean, Choice and Score outputs, adapted from the architecture idea of System One decision models and sized for CPU streaming.
- **A reproducible benchmark protocol** with leakage-safe splits, F-score based evaluation, throughput and latency measurement, and ablations (Sections 5 to 7).

1.1 Problem constraints

ID	Constraint	How the design enforces it
C1	Read-only ingest; no return path, live query or inline block	Ingest opens captures and flow streams read-only; no socket is ever opened toward monitored hosts; alerts carry only a text 'recommended action' that the system never executes.
C2	No payload decryption	TLS/QUIC analysed from cleartext handshake fields, packet sizes, directions and timing only. Payload bytes are never read; QUIC Initial packets are not decrypted either.
C3	Streaming, not batch	Event-time windows with watermarks; 100 ms micro-batches; alerts emitted when a window closes or a detector fires early, so latency is bounded.
C4	Defined throughput target	Targets stated in Section 6.4 and verified by a replay benchmark at stepped offered rates.
C5	Standardized alert schema	Every alert is one JSON record validated against a fixed schema (Section 4.9, Appendix A).

2. Threat Model and What Can Be Observed

The adversary may spoof source addresses, add timing jitter to beacons, use TLS 1.3 or QUIC to hide content, split exfiltration over time, and tunnel data in DNS. The adversary cannot reach the monitoring enclave, because the link is one-way. Table 2 lists, for each family, the passive signal we rely on and what is unavailable because of the architecture.

Threat family	Passive observable	Not available (by design)
a. Volumetric / protocol DDoS (SYN flood, UDP reflection, spoofed floods)	Flow and packet rates; new-flow rate; SYN without completing ACK; source-IP entropy; responses from reflector ports with no matching request on the link; TTL spread	Cannot send SYN cookies, cannot verify a source by reply, cannot rate-limit
b. Botnet C2 beacons	Start times of repeated flows to few destinations; size regularity	Cannot sinkhole or contact the C2 host
c. DGA and DNS tunnelling	Query names, lengths, record types, NXDOMAIN ratio, subdomain diversity per apex domain	Cannot resolve the domain or check registration age
d. Malware in encrypted sessions	JA3/JA3S or JA4 fingerprints, TLS version and extension lists, SNI presence, packet-size and timing sequences	No payload, no decryption, no active certificate fetch

Threat family	Passive observable	Not available (by design)
e. Reconnaissance / port scanning	Fan-out over destination ports and hosts; failed-connection ratio (RST, ICMP unreachable seen on the link)	Cannot ping back or fingerprint the scanner
f. Data exfiltration	Outbound-to-inbound byte ratio; bytes to rarely seen destinations; long-lived uploads	Cannot inspect content or DLP-tag files

3. System Overview

Figure 1 shows the single pipeline. Traffic is processed in three cascaded stages so that cheap, vectorized computation runs on every flow and expensive neural inference runs only where it adds value.

Stage A (every flow): L0 to L4 plus L5a, L5b and L5e. All logic is vectorized and constant-time per flow or per window. **Stage B (relevant records only):** L5c runs on DNS records, L5d on TLS/QUIC sessions. **Stage C (suspicious entities):** L6 to L8 run on entity windows that a Stage A or B detector flagged, plus a small sampled set of normal windows for calibration monitoring. The cascade keeps the neural encoder off the hot path, which is what makes a stated flows-per-second target realistic on a CPU-only host.

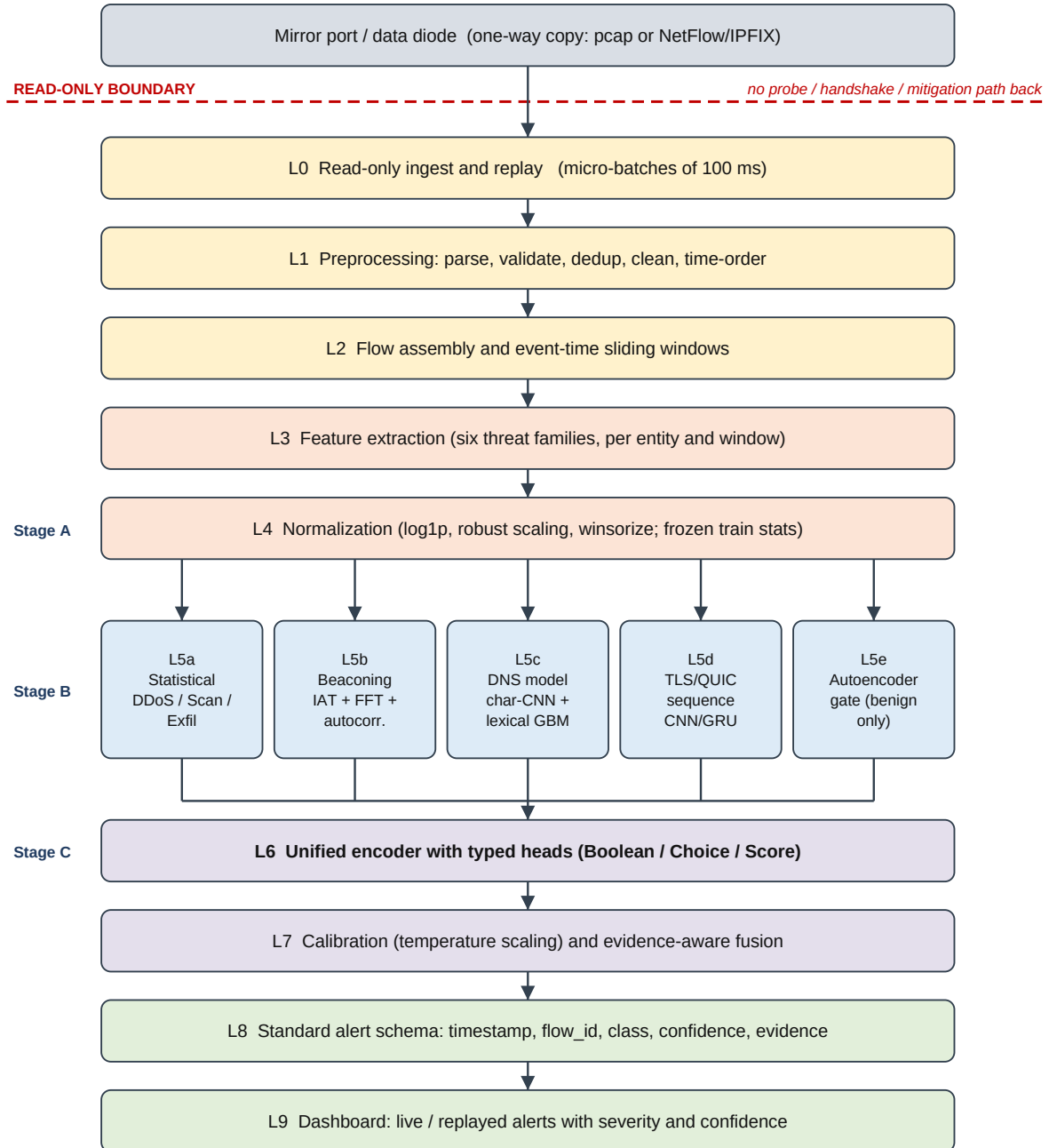


Figure 1. The unified PassiveSentinel pipeline (L0 to L9). The dashed red line marks the one-way boundary: nothing in L0 to L9 can transmit toward the monitored network.

3.1 Latency budget (design allocation)

Stage	Budget (p95)	Mechanism
L0 ingest + micro-batch	100 ms	Fixed 100 ms micro-batch
L1 to L4 preprocessing, features, normalization	200 ms	Vectorized (numpy/polars), per-batch
L5 detectors (A and B)	300 ms	Rules, GBM, ONNX CNN/GRU in batches
L6 to L8 unified encoder, calibration, alert	400 ms	Small encoder, batched on flagged entities
Window closure delay	Window length	1 s (DDoS), 10 s (scan), 60 s (exfil); beaconing needs history
End-to-end target	<= 1 s after window close	Measured in Section 6.5

4. Layer-by-Layer Design

Each layer below is described by its purpose, operation, rationale and output. Layers L1 to L4 are the preprocessing and normalization backbone; their correctness matters as much as the models, since most published intrusion-detection results that fail to reproduce do so because of data handling.

4.1 L0: Read-only ingest and replay

Purpose. Bring the one-directional stream into the enclave without creating any path back.

What it does. Reads pcap files (via a flow exporter such as Zeek, nfdump or CICFlowMeter) or IPFIX/NetFlow/sFlow records from a file, a UDP collector bound to a receive-only interface, or a replay generator that re-emits recorded flows at a configurable rate with original inter-arrival times. Records are pushed to an in-memory bounded queue (optionally Kafka or Redis Streams) in 100 ms micro-batches. A back-pressure counter records drops instead of blocking.

Why this design. A bounded queue with a drop counter makes overload visible and honest (a dropped flow is reported, never hidden), which matters for the throughput claim. Replay with original timing lets the same code run on recorded datasets and live mirrors. Binding only a listening socket and never constructing a client connection makes constraint C1 auditable by inspection.

Output. A stream of raw records with receive timestamp, exporter id and a monotonically increasing sequence number.

4.2 L1: Preprocessing, validation and cleaning

Purpose. Turn raw, messy, duplicated and out-of-order records into a trustworthy event stream.

What it does. (1) **Schema validation:** reject records with missing mandatory fields (time, 5-tuple, counters); count rejects per reason. (2) **Time normalization:** convert all timestamps to UTC epoch milliseconds, keep event time (flow start/end) separate from processing time, correct per-exporter clock offset when a reference is available. (3) **De-duplication:** when a flow is exported by several routers, keep one record using a key of (canonical 5-tuple, start time bucket, exporter-independent counters). (4) **Direction canonicalization:** merge the two unidirectional records of one conversation into a bidirectional flow with 'client' defined as the first sender. (5) **Numeric hygiene:** replace NaN and +/-inf (common in CIC flow CSVs, for example when duration is zero) with a defined sentinel and a binary 'was_missing' flag; clamp negative counters to zero and flag them. (6) **Late-event handling:** a watermark of 5 s tolerates reordering; later records go to a side channel and are counted. (7) **Identifier firewall:** IP addresses, raw ports, flow ids and absolute timestamps are kept only as keys for grouping and for the alert; they are removed from model inputs. (8) **Domain normalization** for DNS: lower-case, strip trailing dot, decode punycode, split the name with a public-suffix list into subdomain, registered domain and suffix.

Why this design. Bad data silently shifts feature distributions. The identifier firewall prevents the single most common source of inflated scores: a model that memorizes the attacker's IP, port or capture time rather than behaviour. The watermark gives bounded latency without waiting forever for stragglers.

Output. A clean, time-ordered stream of canonical bidirectional flows and DNS/TLS side records, plus a data-quality report (reject, duplicate and late counts) shown on the dashboard.

4.3 L2: Flow assembly and event-time sliding windows

Purpose. Create the units that detectors reason about: flows, and aggregates over entities and time.

What it does. Flows use an active timeout of 120 s and an idle timeout of 15 s (configurable). Windows are maintained per *entity*: per destination for DDoS, per source for scan and beaconing, per internal host for exfiltration, per (host, apex domain) for DNS. Window lengths are 1 s (DDoS), 10 s (scan), 60 s (exfiltration), and an expanding history of up to 30 min for beaconing with a minimum of 8 events. Windows slide with a hop of one quarter of their length, so an attack that straddles a boundary is still seen whole. State is held in hash maps with expiry to bound memory.

Why this design. Different threats live at different time scales; one global window would blur a one-second flood or miss a ten-minute beacon. Overlapping windows trade a small compute increase for much lower boundary misses. Expiring state keeps memory bounded under address-space-wide scans.

Output. Per-entity window objects exposing flow lists and incremental counters (unique sets are held in HyperLogLog sketches, entropy counters in count-min sketches, to keep cost constant).

4.4 L3: Feature engineering

Purpose. Convert flows and windows into compact, behaviour-describing numbers that a detector can separate.

What it does. Features are grouped by threat family (Table 3). All are computable passively and incrementally. Entropy features use normalized Shannon entropy; periodicity features are scale-free; ratios use smoothing so that zero denominators are safe.

Why this design. Behavioural, scale-free features generalize across networks better than raw counters, and they double as human-readable evidence in the alert. Sketch-based uniqueness keeps the per-flow cost constant, which is essential for the throughput target.

Output. A fixed-order feature vector per flow (about 40 base features) and per entity-window (family specific), with feature names stored for evidence reporting.

$$H_{\text{norm}}(X) = - \sum_i p_i \cdot \log_2(p_i) / \log_2(N) \quad p_i = \text{count}(x_i) / \text{total}, \quad N = \text{distinct values (range } 0..1)$$
$$CV(IAT) = \text{std}(dt) / \text{mean}(dt) \quad \text{ratio}_{io} = \log((\text{bytes}_{out} + 1) / (\text{bytes}_{in} + 1))$$

Family	Window / key	Core features
Common flow	per flow	duration, fwd/bwd packets and bytes, packet-size mean/std/min/max, inter-arrival mean/std, TCP flag counts (SYN, ACK, FIN, RST, PSH), protocol, port class (well-known / registered / ephemeral), bytes-per-packet, packets-per-second
DDoS	1 s per destination	pps, bps, new-flow rate, distinct sources (HLL), H_{norm} (source IP), SYN share, SYN-without-ACK share, unsolicited-response share from reflector ports (53, 123, 161, 389, 1900, 11211), mean response/request size ratio, TTL variance, small-packet share
Port scan	10 s per source	distinct dst ports, distinct dst hosts, fan-out ratio (ports / flows), failed-connection ratio (RST or ICMP unreachable), mean bytes per flow, sequential-port fraction, H_{norm} (dst port)
Beaconing	30 min per (src, dst, port)	event count, median IAT, CV(IAT), MAD(IAT)/median, dominant FFT frequency and its power share, autocorrelation peak at the dominant lag, CV of bytes per event, series length
DNS (DGA / tunnel)	per query; per (host, apex) 60 s	name length, label count, longest label, H_{norm} of characters, digit and consonant-run ratios, character n-gram log-likelihood under a benign language model, record type (TXT, NULL, CNAME), response size, NXDOMAIN ratio, distinct subdomains per apex, query rate per apex
TLS / QUIC	per session	JA3/JA3S or JA4 hash, TLS version, cipher and extension counts and order class, ALPN, SNI present / length / entropy, first 32 packets as (signed size, log IAT, direction), handshake duration, session byte ratio
Exfiltration	60 s per internal host	ratio _{io} , outbound bytes to first-seen destinations, share of bytes to top destination, upload-session duration, z-score of outbound bytes versus the host's EWMA baseline, burst-versus-sustained index

4.5 L4: Normalization and encoding

Purpose. Put heterogeneous features on comparable scales so models train stably and generalize.

What it does. Rules, in order: (1) **Heavy-tail transform:** apply $\log_{10}$ to counts, bytes, rates and durations. (2) **Robust scaling:** $x' = (x - \text{median}) / \text{IQR}$ using statistics fitted on the *benign training split only*. (3) **Winsorization:** clip to [0.1th, 99.9th] percentile of the training set so one extreme flow cannot dominate. (4) **Bounded features** (entropy, ratios in [0,1]) are left unscaled. (5) **Categoricals:** protocol and port class one-hot; JA3/JA4 and TLS cipher-set hashes go through a frequency-capped embedding with an explicit UNKNOWN bucket (hashes seen fewer than 5 times in training map to UNKNOWN). (6) **Sequences:** packet size divided by MTU with sign for direction, inter-arrival as $\log_{10}(\text{ms})$, padded or truncated to length T with a mask. (7) **Missing flags** from L1 are kept as binary inputs. (8) **Streaming behaviour:** scaler parameters are *frozen* after training and shipped with the model; a drift monitor compares live feature quantiles to the training reference using the Population Stability Index (PSI) and warns when $\text{PSI} > 0.25$.

Why this design. Robust statistics tolerate the heavy tails typical of network data, where a mean/std scaler is dominated by a few floods. Fitting on training data only prevents leakage. Frozen parameters make inference deterministic and reproducible, and drift monitoring tells the operator when retraining is due instead of letting accuracy decay unnoticed.

Output. A normalized float32 tensor per flow or window with a stored scaler artifact (version-hashed together with the model).

```
x_t = log10(x); x_s = clip( (x_t - median_train) / IQR_train , lo_p0.1 , hi_p99.9 )
PSI = sum_b (a_b - e_b) * ln(a_b / e_b)
```

4.6 L5: Specialist detectors

Five detectors run in parallel on the normalized features. Each emits a score in [0,1], a class hypothesis and an evidence vector. Keeping specialists separate preserves explainability and lets each be benchmarked on its own, which also makes the ablation study clean.

L5a: Statistical detectors (DDoS, scan, exfiltration)

Purpose. Fast, explainable detection of rate-, entropy- and fan-out-driven threats.

What it does. A change detector (EWMA baseline with CUSUM) watches pps, new-flow rate and source entropy per destination. Rule-gated gradient-boosted trees (XGBoost or LightGBM) classify SYN flood, reflection/amplification, spoofed flood, horizontal, vertical and block scan, and exfiltration from the family features of Table 3. Rules provide the hard evidence (for example '840 distinct ports in 10 s'); the trees supply a calibrated probability.

Why this design. These threats are defined by simple measurable statistics, so rules plus a small tree model reach high F1 with microsecond cost and full interpretability. A pure deep model here would add latency without adding signal.

Output. Class hypothesis, score, and the triggering feature values.

L5b: Beaconing detector

Purpose. Find malware that calls home at near-regular intervals, even with jitter.

What it does. For each (src, dst, port) series the start times are binned and analysed with the discrete Fourier transform and autocorrelation. A series is beacon-like when $\text{CV}(\text{IAT})$ is low, or when a single frequency holds a large share of spectral power, and the byte counts per event are consistent. A small GBM on these scale-free features outputs the beacon probability; jitter up to about 20 percent of the period is covered by training augmentation (random timing jitter).

Why this design. Periodicity is a structural property best captured by spectral and autocorrelation statistics; scale-free features let one model cover 5-second and 1-hour beacons alike.

Output. Beacon probability, estimated period, number of events, regularity metrics.

L5c: DNS model (DGA and tunnelling)

Purpose. Separate algorithmically generated or data-carrying names from human-chosen names.

What it does. Two models are fused. (1) A **character-level CNN** (embedding 32, three conv widths 3/4/5 with 64 filters, global max-pool, dense 64) over the registered-domain string classifies DGA versus benign and, optionally, DGA family. (2) A **lexical-behavioural GBM** uses entropy, n-gram likelihood, length, record type, NXDOMAIN ratio and distinct subdomains per apex to catch tunnelling and dictionary-DGA that look lexically plausible. Training negatives come from a popularity list (Tranco); positives from public DGA feeds and DGArchive-style family generators.

Why this design. Characters carry the signal for random DGA; behaviour carries the signal for dictionary-DGA and tunnelling. The CNN is a few tens of thousands of parameters, fast on CPU, and every score has a readable justification (high entropy, long label, many unique subdomains).

Output. DGA probability, tunnelling probability, supporting lexical and behavioural features.

L5d: TLS/QUIC sequence model (metadata only)

Purpose. Detect malware inside encrypted sessions without touching payload.

What it does. Input: the first 32 packets as a (signed size, log IAT, direction) sequence plus the handshake metadata. A **1D-CNN or GRU** encodes the sequence; fingerprint embeddings (JA3/JA4, JA3S) and numeric handshake features are concatenated; a dense head outputs malicious probability and, optionally, malware family. JA4 is preferred over JA3 because modern clients randomize extension order, which breaks JA3 stability. For QUIC, which hides most handshake fields, the model relies on packet-size and timing sequences and flow-level metadata; QUIC Initial packets are not decrypted.

Why this design. Packet-length and timing sequences of the encrypted exchange expose the protocol's behaviour (for example fixed-size check-in then download) even when content is hidden, and are standard in encrypted-malware research. Fingerprint features add precise known-bad evidence but are dataset-specific, so they are ablated in Section 7.

Output. Malicious probability, family hypothesis, top contributing sequence positions and handshake attributes.

L5e: Autoencoder anomaly gate (unknown threats)

Purpose. Catch behaviours not covered by the supervised detectors.

What it does. A dense autoencoder (input ~40, hidden 32-16-8-16-32, output ~40) is trained on *benign flows only*. The anomaly score is the per-feature-normalized reconstruction error. The alert threshold is the 99.5th percentile of benign validation error (adjustable to a target false-positive rate). An Isolation Forest is kept as a baseline.

Why this design. Supervised models only know the attacks in their training data; a one-class model trained on normal behaviour flags novelty. It is small enough to run on every flow.

Output. Anomaly score and the features with the largest reconstruction error (evidence).

4.7 L6: Unified encoder with typed heads (Jev-inspired)

Purpose. Fuse every specialist's output and the raw window context into one coherent, calibrated decision, and answer several typed questions at once. This is the single place where the pipeline reasons across detectors, so that, for example, a periodic flow to a rare destination *and* an elevated autoencoder score *and* a bad TLS fingerprint become one high-confidence C2 finding instead of three separate weak alerts.

Relation to Jev. TypeSafe AI's Jev is described publicly as a System One model that maps unstructured state to typed probabilistic decisions: a bidirectional (non-causal) encoder produces a contextual state memory; a pool of typed questions (Boolean, Choice, Score) attends over that memory in parallel; and separate heads output a probability, a probability distribution, or a score expectation, each with a confidence. We adopt this *architectural idea* at a much smaller scale. We do not use the Jev service or weights (it is a hosted, waitlisted API, and calling it would violate the enclave constraint), we do not claim parity with it, and we train with standard supervised multi-task losses plus post-hoc calibration, not the vendor's proprietary training method. Vendor performance figures are self-reported and are not used anywhere in our claims.

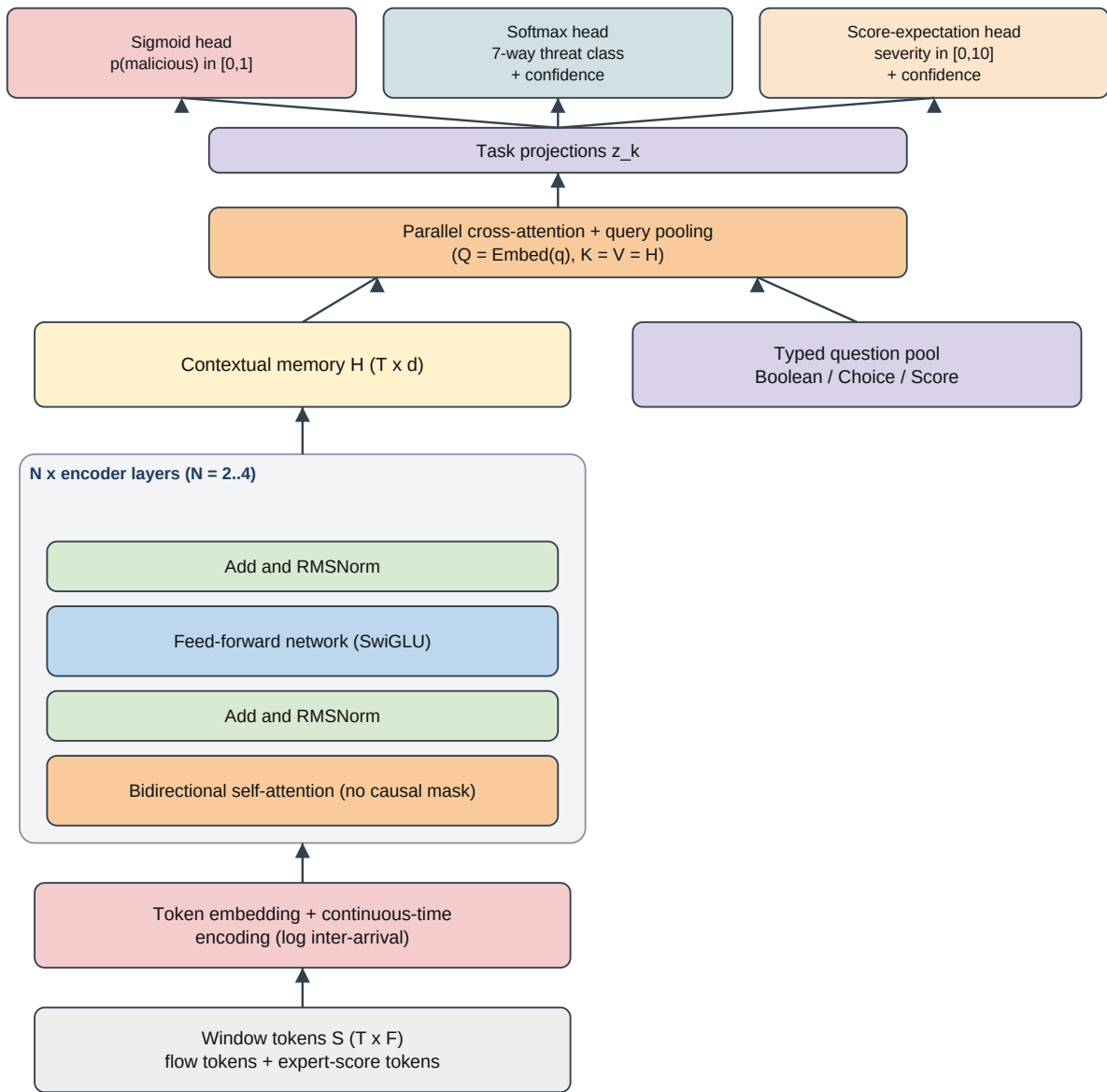


Figure 2. L6 architecture: a small bidirectional encoder builds a contextual memory of the entity window; typed questions attend over it in parallel and feed three heads.

What it does. The input is a sequence of $T = 64$ tokens for one entity: up to 63 recent flow tokens (normalized features from L4) plus one *expert-score token* that packs the outputs of L5a to L5e (scores, class hypotheses, periodicity statistics). Tokens receive a linear embedding and a continuous-time encoding of log inter-arrival time (a stand-in for positional encoding that respects real timing; rotary position encoding is an alternative). $N = 2$ encoder layers (up to 4) apply bidirectional self-attention (no causal mask, because the window is complete when scored), RMSNorm and a SwiGLU feed-forward block, with $d_{\text{model}} = 64$, four heads and $d_{\text{ff}} = 256$, roughly 0.15 to 0.5 M parameters depending on N . The resulting memory H is read by a typed question pool: each question embedding is a query in a parallel cross-attention that pools H into a task projection z_k . Three head types follow:

- **Boolean head** (sigmoid): `is_malicious`, `is_spoofed_source`, `is_periodic`, `is_encrypted_threat`; each returns p in $[0,1]$.
- **Choice head** (softmax): `threat_class` over `{benign, ddos, beaconing, dga_tunnel, encrypted_malware, recon_scan, exfiltration}`; returns a distribution and the top-class confidence.
- **Score head** (expectation over $K = 11$ bins): severity in $[0,10]$ as the expectation of a bin distribution, so the model also exposes uncertainty of the score. Training labels for severity are *weak labels* produced by the deterministic function in Section 4.9, because public datasets contain no analyst severity; this head is therefore reported as secondary.

Training objective. $L = L_{\text{choice}} + 0.5 * L_{\text{boolean}} + 0.2 * L_{\text{score}}$, with focal loss ($\gamma = 2$) and class weights in L_{choice} for rare classes, and label smoothing 0.05. Entities are labelled from ground truth by overlap with attack intervals.

Why this design. (1) One shared encoder with several typed heads is a multi-task setup that regularizes each task and gives consistent outputs. (2) Typed outputs match what the analyst needs: a yes/no flag, a class with confidence, and a severity. (3) Bidirectional attention uses the whole window rather than only the past, which suits complete windows. (4) Because it runs only on flagged entities (Stage C), its cost does not limit the flow-rate target. (5) The expert-score token lets interpretable specialists feed the neural fuser, so a failure of one specialist is visible and can be ablated. **Output.** Boolean probabilities, a class distribution and a severity distribution per entity window.

4.8 L7: Calibration and evidence-aware fusion

Purpose. Make confidence scores mean what they say, and produce one final decision per event.

What it does. Logits of the Choice and Boolean heads are divided by a temperature T fitted on the validation set (temperature scaling) to minimize negative log-likelihood; Expected Calibration Error (ECE) is reported before and after. Final confidence is the calibrated probability of the chosen class. If a specialist and L6 disagree, the alert keeps the L6 class but lists the specialist's dissenting score in its evidence, and confidence is capped at the lower of the two when the specialist is above its own threshold. Events within a merge interval for the same entity and class are coalesced into one incident with a first-seen and last-seen time.

Why this design. An analyst triaging alerts needs confidence they can trust: a score of 0.9 should be right about 90 percent of the time. Temperature scaling is simple, preserves the ranking and is known to reduce miscalibration of neural networks. Merging avoids alert storms during long attacks.

Output. A calibrated confidence, final class, and a coalesced incident record.

4.9 L8: Alert schema, severity and evidence

Purpose. Emit structured, machine-readable intelligence in a fixed schema. **What it does.** Each alert contains the fields in Table 4. Severity is a deterministic, auditable function of confidence, impact and persistence; the L6 score head is trained to mimic it and may refine it. **Evidence** is the top-k features ranked by deviation from the benign baseline (robust z-score), plus the specialist-specific facts (period, ports, domain, fingerprint). **Why.** A fixed schema makes the output consumable by SIEMs and satisfies constraint C5; auditable severity avoids black-box prioritization. **Output.** One JSON record per incident (example in Appendix A).

```
severity = 10 * ( 0.5*confidence + 0.3*impact + 0.2*persistence )
impact = min(1, log10(1+volume)/log10(1+V_ref))
```

Field	Type	Meaning
alert_id, schema_version	string	Unique id and schema version for compatibility
timestamp	ISO-8601 UTC	Alert emission time; first_seen and last_seen carry event time
flow_id	string	Stable id of the triggering flow, or entity key for window-level alerts
threat_class	enum	ddos beaconing dga_tunnel encrypted_malware recon_scan exfiltration anomaly
confidence	float 0..1	Calibrated probability of the class
severity	enum + score	low medium high critical, with the numeric 0..10 score
evidence	object	Supporting features with values, baselines and deviations; detector scores
recommended_action	string	Text for the analyst only; never executed by the system
latency_ms	int	Alert time minus window-close time, for audit of bounded latency

4.10 L9: Dashboard

Purpose. Let analysts see and triage detections live or in replay.

What it does. FastAPI serves a WebSocket stream of alerts and a REST endpoint for history; a Streamlit or React front end shows a live alert feed, severity and confidence badges, a per-class time series, top talkers, an evidence

panel for the selected alert, data-quality counters (rejects, drops, late records), measured throughput and latency, and a replay control with speed selection.

Why this design. Surfacing quality and performance counters next to alerts keeps the system honest: an analyst can tell a quiet network from a collector that is dropping records.

Output. A browser view; alerts are also appended to a rotating JSON-lines file and a DuckDB/SQLite table for forensics.

5. Training and Validation Approach

5.1 Datasets

Purpose	Candidate public data	Notes
DDoS, scan, bot, general benign	CIC-IDS2017 / CSE-CIC-IDS2018; UNSW-NB15	CIC-IDS2017 has known labelling and flow-extraction problems; use a corrected re-extraction when available and document the version
Botnet beaconing	CTU-13	Real botnet captures with background traffic
DGA / DNS	Tranco top list (benign); public DGA feeds and DGArchive-style generators	Check licences; split by DGA family for the generalization test
DNS tunnelling / exfiltration	CIC-Bell-DNS-EXF-2021 (verify availability); synthetic injection	Synthetic exfiltration is injected into benign background with documented parameters
Encrypted malware	Stratosphere/CTU malware captures with TLS; public malware-traffic PCAP archives	Use only metadata; label by capture provenance
Spoofed floods, reflection	Synthetic flow generator in the offline simulator	Parameterized patterns injected into background; parameters logged for reproducibility

5.2 Splits and leakage control

- **Split before windowing.** Divide the capture into train, validation and test by time (for example train Monday to Wednesday, validate Thursday morning, test Thursday afternoon and Friday for CIC-IDS2017), and leave a purge gap at least as long as the longest window between splits, so no window straddles two splits.
- **Group by attacker.** All records of one attacker source stay in one split; otherwise the model can memorize a source.
- **Group by family.** For DGA and malware, keep entire families out of training for a family-held-out test of generalization.
- **Fit-on-train only.** Scalers, vocabularies, thresholds' reference data and the benign language model are fitted on the training split. Calibration temperature and operating thresholds use the validation split only. The test split is touched once per final configuration.
- **Cross-dataset test.** Train on one corpus and test on another (class-mapped), reporting the drop honestly; this is the realistic estimate of field performance.
- **Seeds and intervals.** Five seeds; report mean and standard deviation, and bootstrap 95 percent confidence intervals for F1.

5.3 Training procedure

Specialists are trained first. GBMs use early stopping on validation macro-F1; the DNS CNN and TLS sequence model use AdamW, learning rate 1e-3 with cosine decay, weight decay 1e-2, batch 256, up to 30 epochs with early stopping; the autoencoder is trained on benign data with MSE. The unified encoder (L6) is trained last on entity windows whose expert-score tokens come from *out-of-fold* predictions of the specialists, so that L6 never sees scores that were produced on the specialists' own training data (a standard stacking safeguard). Hyper-parameters are tuned by grouped cross-validation on the training period only (Optuna or random search, fixed budget). Initial defaults are in Appendix B.

6. Benchmark Protocol

This section defines the benchmark so that every number in the final paper is measurable and repeatable. Result tables are templates: they are to be filled by the repository's benchmark scripts, and no result is reported in this draft.

6.1 Detection metrics

- **Per-class precision, recall and F1** (flow or window level). $F1 = 2PR/(P+R)$; macro-F1 averages classes equally and is the headline metric because attack classes are rare; weighted-F1 and MCC are secondary.
- **PR-AUC** per class (more informative than ROC-AUC under heavy imbalance).
- **False-positive rate and alert volume**: benign flows flagged, and alerts per hour on benign-only replay, since analyst workload is the real cost of false positives.
- **Episode-level recall**: an attack episode counts as detected if at least one correct-class alert falls within it; **time-to-detect**: first correct alert time minus episode start (p50, p95).
- **Calibration**: ECE with 15 bins and Brier score, before and after temperature scaling.

6.2 Operating points

Thresholds are chosen on the validation set in two ways and both are reported: (a) the threshold that maximizes per-class F1, and (b) the threshold that fixes the benign false-positive rate at 0.1 percent. The test set is then evaluated at those fixed thresholds without re-tuning.

6.3 Hardware and software record

Item	Value to record
CPU model, cores, frequency	[fill]
RAM	[fill]
OS and kernel	[fill]
Python, library versions, ONNX Runtime version	[fill]
Run mode	single process / N workers; GPU used: yes/no
Input mode	flow records (IPFIX/NetFlow) or pcap via exporter (state which)
Random seeds and dataset versions / hashes	[fill]

6.4 Throughput and latency targets (design goals, to be verified)

Metric	Target	How it is measured
Sustained flow-record rate (single commodity node, CPU only)	$\geq 20,000$ flows/s	Replay at 1k, 5k, 10k, 20k, 30k, 50k flows/s for 10 min each; 'sustained' = zero queue drops and p95 latency within budget
Sustained raw traffic via exporter (if pcap mode is demonstrated)	≥ 200 Mbps (state packet size mix)	Replay with tcpreplay at stepped Mbps; report exporter and pipeline limits separately
Alert latency (window close to alert emitted)	p95 ≤ 1 s, p99 ≤ 2 s	latency_ms field, measured in the replay clock
Memory	≤ 4 GB at target rate	Peak RSS over the run
Macro-F1, in-distribution test	≥ 0.95	Section 6.1 on the temporal test split
Per-class F1: DDoS, scan	≥ 0.95	Same
Per-class F1: beaconing, DGA	≥ 0.90	Same
Per-class F1: encrypted malware, exfiltration	≥ 0.85	Same
Macro-F1, cross-dataset	≥ 0.75 (report drop honestly)	Train on corpus A, test on corpus B
ECE after calibration	≤ 0.05	15-bin ECE on test
Benign alert volume	[set per site] alerts/hour	Benign-only replay

These are goals chosen to be ambitious but plausible for well-separated classes; the cross-dataset and encrypted-malware targets are the most likely to fall short, and the paper must report whatever is measured.

6.5 Latency measurement method

Each micro-batch is stamped on entry (t_{in}). Alert latency is t_{emit} minus the closing time of the window that triggered the alert, measured on the replay clock so that faster-than-real-time replay does not distort it. Component timings (L0 to L8) are logged per batch to show where the budget is spent. The benchmark script also records queue depth and the drop counter; a run with any drops is marked as exceeding the sustained rate.

6.6 Result templates

Class	Precision	Recall	F1	PR-AUC	Episode recall	TTD p50 / p95 (s)
DDoS	[]	[]	[]	[]	[]	[]
Beaconing	[]	[]	[]	[]	[]	[]
DGA / DNS tunnelling	[]	[]	[]	[]	[]	[]
Encrypted malware	[]	[]	[]	[]	[]	[]
Recon / scan	[]	[]	[]	[]	[]	[]
Exfiltration	[]	[]	[]	[]	[]	[]
Macro average	[]	[]	[]	[]	[]	[]

Offered rate (flows/s)	Achieved	Drops	p50 latency	p95 latency	p99 latency	CPU %	RSS (MB)
1,000	[]	[]	[]	[]	[]	[]	[]
5,000	[]	[]	[]	[]	[]	[]	[]
10,000	[]	[]	[]	[]	[]	[]	[]
20,000	[]	[]	[]	[]	[]	[]	[]
50,000	[]	[]	[]	[]	[]	[]	[]

7. Ablation Study Design

Ablations show which layer earns its place. Each row removes or changes one element and reports macro-F1, per-class F1 and latency.

ID	Configuration	Question answered
A0	Rules / statistical only (L5a, L5b)	Baseline: how far do simple statistics go?
A1	A0 + specialist ML (L5c, L5d)	Value of ML where rules cannot see (DGA, encrypted)
A2	A1 + autoencoder gate (L5e)	Detection of unknown or novel behaviour; false-positive cost
A3	A2 + unified encoder, typed heads (L6) and calibration (L7)	Value of fusion and calibrated confidence (main claim)
A4	A3 without expert-score token	Does L6 add value beyond the specialists, or only repackage them?
A5	A3 with a plain MLP head instead of encoder	Value of attention over the window
A6	A3 without robust scaling / log1p	Effect of normalization (L4)
A7	A3 with identifier leakage allowed (IPs, ports, time)	Quantifies leakage inflation; report to show honesty
A8	A3 without JA3/JA4 fingerprint features	Dependence on dataset-specific fingerprints
A9	A3 without temperature scaling	Calibration benefit (ECE)
A10	Single-head (choice only) versus multi-head training	Value of typed multi-task outputs

8. Additional Treatments Considered

Beyond the core layers, the following treatments address recurring failure modes in network ML. Each is scoped to a layer so that it can be switched on and evaluated independently.

Issue	Treatment	Layer
Class imbalance	Class weights, focal loss, window-level oversampling of rare attacks; never random oversampling of overlapping windows across splits	L5, L6

Issue	Treatment	Layer
Redundant features	Drop one of any pair with $ r > 0.95$; mutual-information ranking; SHAP on GBMs to prune low-value features	L3
Label noise	Audit mislabelled flows (known in CIC-IDS2017), use re-extracted labels, report on the corrected subset	Data
Concept drift	PSI monitor on inputs; periodic recalibration of temperature; scheduled retraining on a sliding benign window	L4, L7
Adversarial evasion	Timing-jitter and packet-padding augmentation; evaluate with perturbed replays; do not rely on a single signal	L5b, L5d
Explainability	Rule triggers, GBM SHAP, autoencoder per-feature error, attention weights as supporting (not proof) evidence	L8
Encrypted protocol change	Prefer JA4 over JA3; treat TLS 1.3 and ECH as reducing handshake visibility; rely more on sequences	L5d
Spoofing	Source entropy plus TTL variance plus unsolicited-response share, because entropy alone also rises in legitimate flash crowds	L5a
Cold start	Per-host baselines start from a global prior and move to host-specific as data accrues	L5a
Privacy / forensics	Optional keyed hashing of internal IPs in stored evidence; raw records kept in the immutable capture	L8

9. Limitations and Threats to Validity

- **Dataset realism.** Public datasets are lab-generated and dated; results will not transfer unchanged. The cross-dataset test and a replay of a site-specific benign capture are the guard.
- **Label quality.** Flow labels in public corpora contain errors, which bounds attainable F1 and can reward models that learn artefacts.
- **Encrypted traffic.** TLS 1.3, ECH and QUIC reduce visible metadata; detection of encrypted malware may fall as protocols evolve.
- **Passive-only limits.** Without active verification, some confirmations (for example whether a domain is truly malicious) are impossible; alerts are leads for an analyst, not verdicts.
- **Synthetic attacks.** Injected attack traffic reflects our assumptions about attackers; parameters are logged and varied to reduce this bias.
- **Severity labels.** The severity head is trained on weak, rule-derived labels, so it is a prioritization aid, not an independent judgment.
- **Scale of the encoder.** The L6 encoder is deliberately small; claims about it should not be extended to large foundation-scale models.

10. Repository Layout and Reproducibility

```

passive-sentinel/
  ingest/          readers, replayer, bounded queue (L0)
  preprocess/      validate, dedup, time, clean, domain parsing (L1)
  windows/         flow assembly, sketches, windows (L2)
  features/        per-family extractors + feature registry (L3)
  normalize/       scaler fit/freeze, encoders, drift monitor (L4)
  detectors/       stat/, beacon/, dns/, tls/, autoencoder/ (L5)
  fusion/          unified encoder, heads, calibration (L6-L7)
  alerts/          schema (JSON Schema), severity, evidence, sinks (L8)
  dashboard/       FastAPI + WebSocket + Streamlit/React (L9)
  bench/           datasets, splits, metrics, throughput, latency, ablations
  docs/            model cards, feature dictionary, this paper
  configs/         one YAML per experiment (seeds, windows, thresholds)

```

Reproducibility: every experiment is one YAML config; datasets are referenced by hash; the scaler, model weights and thresholds are version-hashed together; and the benchmark scripts regenerate every table and figure from raw logs. A Docker image pins library versions.

11. Conclusion

PassiveSentinel treats the one-way, no-decryption, streaming constraints as architecture, not afterthought. Cheap statistical detectors cover rate-, entropy- and fan-out-defined threats; targeted ML covers DGA and encrypted sessions; an autoencoder watches for novelty; and a compact Jev-inspired encoder with typed heads fuses everything into calibrated alerts with evidence. The benchmark protocol is designed so that each claim can be verified or refuted, and so that the cost of each layer is visible. The next step is to run the protocol and fill the result templates; the ablation table then states precisely which layers justify their cost.

References

Note: verify every reference (authors, venue, year, availability) before submission.

- [1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP, 2018 (CIC-IDS2017).
- [2] G. Engelen, V. Rimmer, W. Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. IEEE Security and Privacy Workshops, 2021.
- [3] S. Garcia, M. Grill, J. Stiborek, A. Zunino. An empirical comparison of botnet detection methods. Computers and Security 45, 2014 (CTU-13).
- [4] N. Moustafa, J. Slay. UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS, 2015.
- [5] B. Claise, B. Trammell, P. Aitken. Specification of the IP Flow Information Export (IPFIX) Protocol. RFC 7011, 2013.
- [6] J. Althouse et al. JA3: a method for profiling SSL/TLS clients (Salesforce, 2017); FoxIO. JA4+ network fingerprinting, 2023.
- [7] B. Anderson, D. McGrew. Identifying encrypted malware traffic with contextual flow data. ACM AISec, 2016.
- [8] D. Plohmann et al. A comprehensive measurement study of domain generating malware. USENIX Security, 2016.
- [9] V. Le Pochat et al. Tranco: a research-oriented top sites ranking hardened against manipulation. NDSS, 2019.
- [10] A. Lakhina, M. Crovella, C. Diot. Mining anomalies using traffic feature distributions. ACM SIGCOMM, 2005.
- [11] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai. Kitsune: an ensemble of autoencoders for online network intrusion detection. NDSS, 2018.
- [12] F. T. Liu, K. M. Ting, Z.-H. Zhou. Isolation forest. IEEE ICDM, 2008.
- [13] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. On calibration of modern neural networks. ICML, 2017.
- [14] T.-Y. Lin et al. Focal loss for dense object detection. ICCV, 2017.
- [15] A. Vaswani et al. Attention is all you need. NeurIPS, 2017.
- [16] B. Zhang, R. Sennrich. Root mean square layer normalization. NeurIPS, 2019.
- [17] N. Shazeer. GLU variants improve Transformer. arXiv:2002.05202, 2020 (SwiGLU).
- [18] J. Su et al. RoFormer: enhanced transformer with rotary position embedding. Neurocomputing, 2024.
- [19] J. Davis, M. Goadrich. The relationship between precision-recall and ROC curves. ICML, 2006.
- [20] TypeSafe AI. Jev and System One models: public announcement and website (typesafe.ai), 15 September 2026. Vendor description only; proprietary, hosted, waitlisted; not used in this work.

Appendix A. Example alert record

```
{
  "schema_version": "1.0",
  "alert_id": "a-000123",
  "timestamp": "2026-10-01T14:02:11.480Z",
  "first_seen": "2026-10-01T14:02:01.000Z",
  "last_seen": "2026-10-01T14:02:11.000Z",
  "flow_id": "src:10.2.3.4|win:10s|t:1759327321",
  "threat_class": "recon_scan",
  "confidence": 0.93,
  "severity": {"level": "high", "score": 7.8},
  "evidence": {
    "distinct_dst_ports": {"value": 840, "baseline": 6, "robust_z": 41.2},
    "failed_conn_ratio": {"value": 0.91, "baseline": 0.04},
    "sequential_port_fraction": {"value": 0.77},
    "detector_scores": {"stat": 0.97, "autoencoder": 0.81, "unified": 0.93}
  },
  "recommended_action": "Analyst review; consider blocking source at perimeter (text only)",
  "latency_ms": 412
}
```

Appendix B. Initial hyper-parameter defaults (tunable)

Component	Setting
Windows	DDoS 1 s, scan 10 s, exfil 60 s, beacon history 30 min (min 8 events); hop = window / 4
Flow timeouts / watermark	active 120 s, idle 15 s; watermark 5 s; micro-batch 100 ms
GBM (L5a, L5b, DNS lexical)	300 trees, depth 6, learning rate 0.05, subsample 0.8, early stopping
DNS char-CNN	embedding 32; conv widths 3/4/5 x 64 filters; global max-pool; dense 64; dropout 0.2
TLS/QUIC sequence model	T = 32 packets x 3 channels; 1D-CNN (3 blocks, 64 channels) or GRU(64); fingerprint embedding 16
Autoencoder	40-32-16-8-16-32-40, ReLU, MSE; threshold = 99.5th percentile of benign validation error
Unified encoder (L6)	T = 64 tokens; d_model 64; 4 heads; N = 2; d_ff 256 (SwiGLU); RMSNorm; dropout 0.1; K = 11 score bins
Loss	choice (focal, gamma 2, class weights) + 0.5 boolean + 0.2 score; label smoothing 0.05
Optimizer	AdamW, lr 1e-3 cosine, weight decay 1e-2, batch 256, up to 30 epochs, early stop on validation macro-F1
Calibration	Temperature scaling on validation; ECE 15 bins
Drift	PSI per feature on 10 quantile bins; warn at 0.25